

Analysis Report

Like

Share

860 people like this. [Sign Up](#) to see what your friends like.

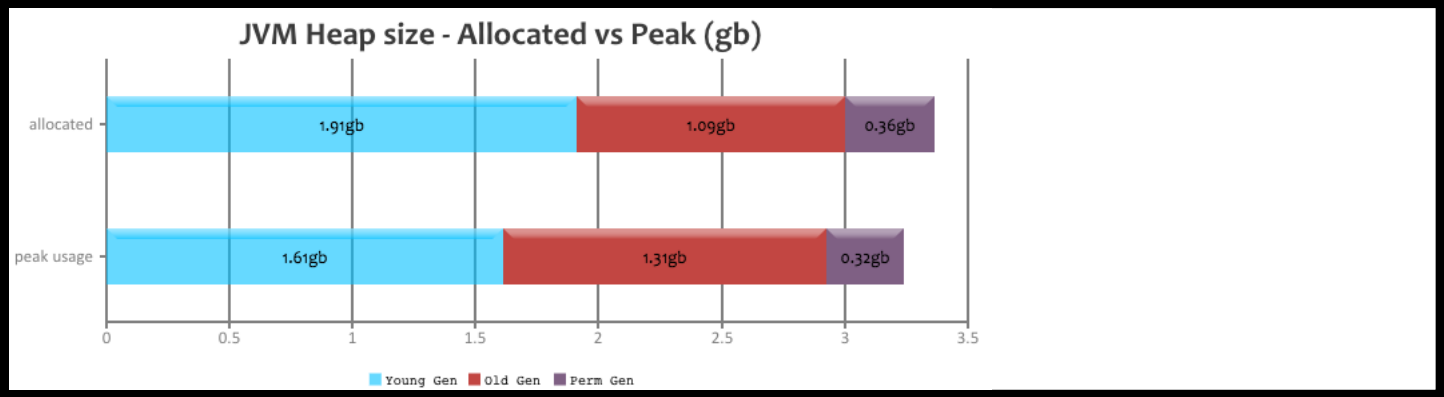
Share this Report

GC log file: **gc-2017_08_04-14_47.log.0**

Duration: **6 min 33 sec 129 ms**

JVM Heap Size

Generation	Allocated ?	Peak ?
Young Generation	1.91 gb	1.61 gb
Old Generation	1.09 gb	1.31 gb
Perm Generation	372 mb	322.86 mb
Young + Old + Perm	3.36 gb	3.21 gb



Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/) (https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

1 **Throughput ?** : **97.708%**

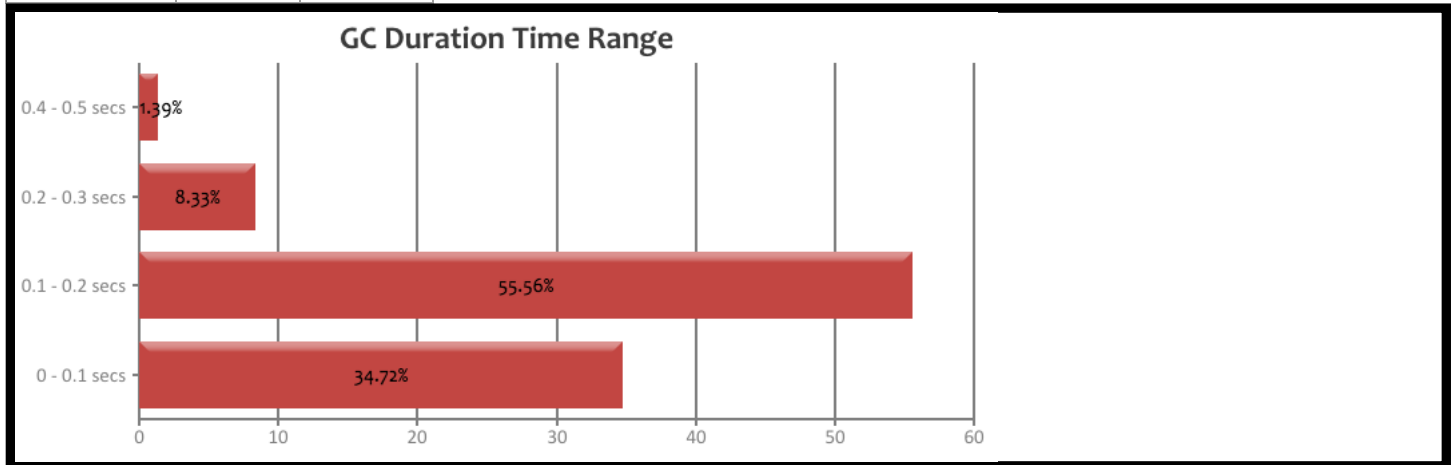
2 **Latency:**

Avg GC Time ?	109 ms
Max GC Time ?	470 ms

GC Duration Time Range ?:

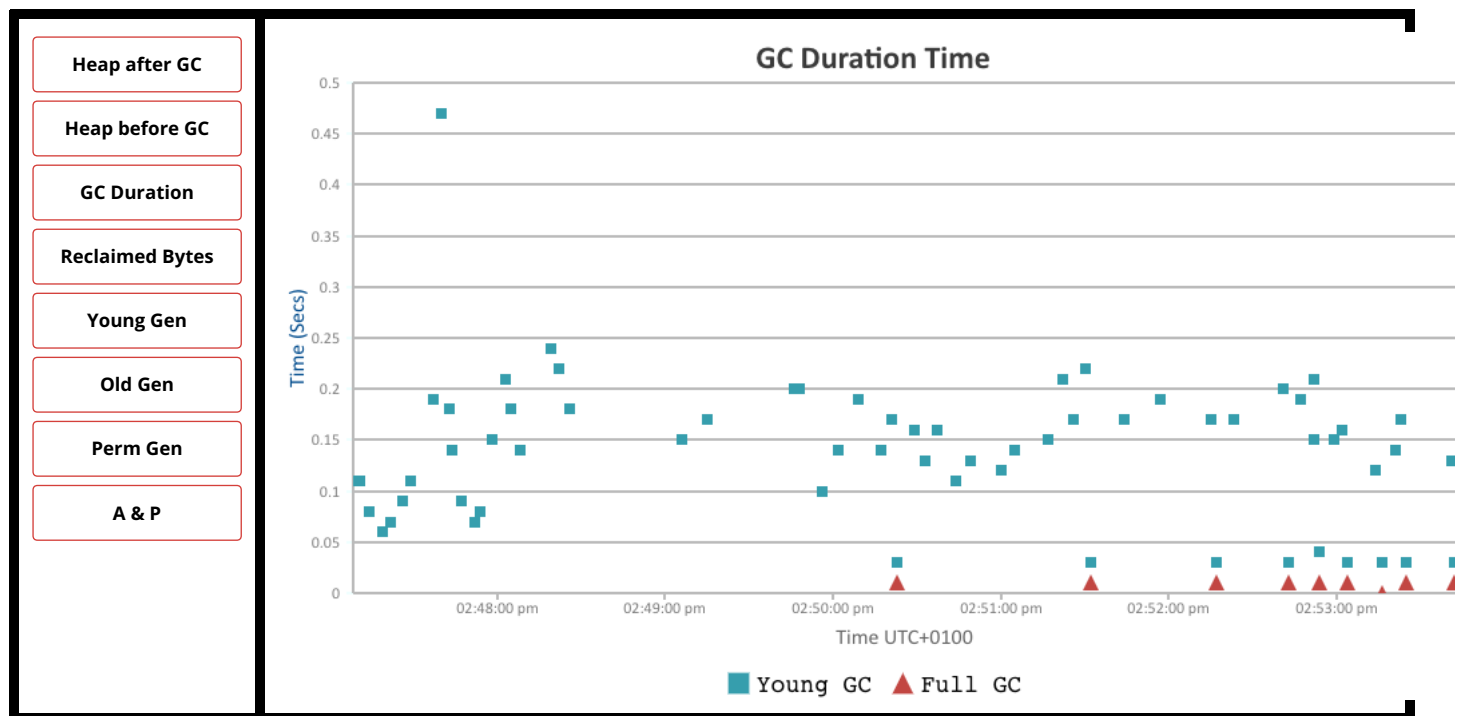
Duration (secs)	No. of GCs	Percentage
0 - 0.1	25	34.722%
0.1 - 0.2	40	90.278%

0.2 - 0.3	6	98.611%
0.4 - 0.5	1	100.0%

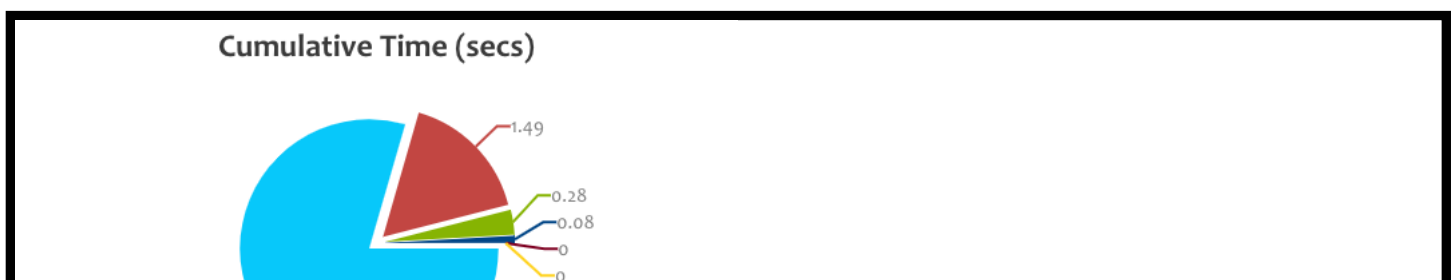


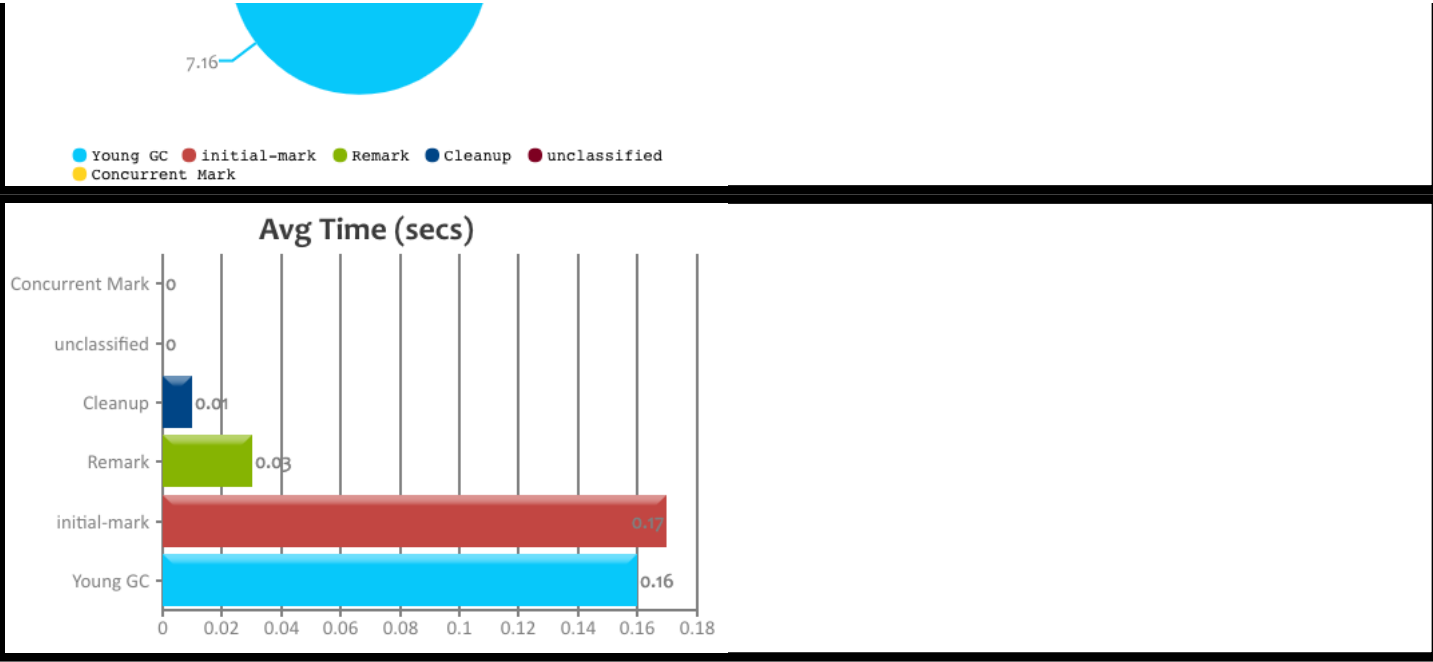
Interactive Graphs

(All graphs are zoomable)



G1 GC Statistics





	unclassified	Concurrent Mark	Cleanup	Young GC	Remark	initial-mark	Total
Count ⓘ	1	9	9	46	9	9	83
Total GC Time ⓘ	0	0	80 ms	7 sec 160 ms	280 ms	1 sec 490 ms	9 sec 10 ms
Avg GC Time ⓘ	0	0	9 ms	156 ms	31 ms	166 ms	109 ms
Avg Time std dev	0	0	3 ms	64 ms	3 ms	29 ms	84 ms
Min/Max Time ⓘ	0 / 0	0 / 0	0 / 10 ms	0 / 470 ms	0 / 40 ms	0 / 220 ms	0 / 470 ms
Avg Interval Time ⓘ	n/a	24 sec 952 ms	24 sec 952 ms	8 sec 258 ms	24 sec 952 ms	24 sec 959 ms	15 sec 197 ms

⚙ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	45.77 gb
Total promoted bytes ⓘ	633 mb
Avg creation rate ⓘ	119.22 mb/sec
Avg promotion rate ⓘ	1.61 mb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are 8 flavours of [OutOfMemoryErrors](https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf) (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them(Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

⌵ Consecutive Full GC ⓘ

None.

■ Long Pause ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

❓ GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Not reported in the log.

⌘ Tenuring Summary ⓘ

Not reported in the log.

📄 Command Line Flags ⓘ

Not reported in the log.

🏆 Become a DevOps champion in your organization

(Best practises/tools)

- ✓ Use **fastThread.io** (<https://fastthread.io/>) to analyze thread dumps, core dumps and hs_err_pid files
 - ✓ Do proactive Garbage Collection analysis on all your JVMs (not just one or two) [using the GC log analysis API](https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/) ⓘ (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)
 - ✓ Always [rotate the GC log files](https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) (https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) to prevent critical garbage collection data loss
 - ✓ Use '[Pro](http://gceasy.io/pricing.jsp)' version (http://gceasy.io/pricing.jsp) for 10x fast, unlimited, secure usage
-

Do you like this report?